

Projet DATA - M2 MIAAGE APPRENTISSAGE

Simulateur de portefeuille passif ETF, DCA et régression linéaire

Compte rendu de projet

MEITE MALIKA SINAI
GEORGIAKAKIS OLYMPIA
BOUZIDI ABDESSALEM

Sommaire

I. Introduction	p. 2
II. Notions financières	p. 3
A. ETF et investissement passif	p. 3
B. DCA : Dollar Cost Averaging	p. 4
C. TER	p. 4
D. Backtesting	p. 4
E. Régression linéaire appliquée à un ETF	p. 5
III. Architecture et choix techniques	p. 5
1. Méthode de travail	p. 5
2. Architecture générale	p. 6
3. Modèle de données	p. 7
IV. Pipeline de données	p. 8
1. Sources utilisées	p. 8
2. Processus ETL	p. 9
3. Qualité et cohérence des données	p. 9
4. Fréquence de mise à jour	p. 9
V. Simulateur DCA	p. 10
1. Objectif du module	p. 10
2. Algorithme de calcul	p. 10
3. Sauvegarde des résultats	p. 11
4. Cas de test	p. 11
5. Analyse de l'impact des frais	p. 13
VI. Régression linéaire	p. 14
1. Objectif du module	p. 14
2. Méthodologie statistique	p. 14
3. Indicateurs interprétés	p. 15
4. Résultats sur deux ETF	p. 15
5. Interprétation critique	p. 17
6. Limites du modèle de régression	p. 18
VII. Conclusion	p. 18
Annexes	p. 19

I. INTRODUCTION

L'investissement passif prend une place croissante dans les stratégies d'épargne de long terme. Les ETF permettent d'investir dans un indice entier, comme le MSCI World ou le S&P 500, sans sélectionner soi-même les actions. Cette approche est souvent associée à des frais réduits, à une forte diversification et à une logique de long terme.

Dans ce contexte, notre projet consiste à développer une application web permettant de manipuler des données historiques d'ETF et de comprendre concrètement l'effet d'une stratégie d'investissement programmée. L'utilisateur peut explorer des ETF, lancer une simulation DCA et analyser une tendance historique à l'aide d'une régression linéaire.

Dans ce contexte, le projet s'est construit autour d'un objectif principal : concevoir un outil à la fois simple, lisible et fiable permettant de mieux comprendre l'investissement passif. L'application doit permettre de comparer plusieurs ETF et d'illustrer concrètement l'effet du temps, des versements réguliers et des frais de gestion sur la performance d'un portefeuille.

Le projet n'a donc pas vocation à fournir un conseil financier personnalisé. Il s'inscrit plutôt dans une démarche pédagogique de backtesting : à partir de données historiques, il rejoue des scénarios passés afin d'aider l'utilisateur à comprendre les mécanismes financiers à l'œuvre. Les résultats obtenus doivent ainsi être interprétés avec prudence, puisqu'une performance passée ne garantit jamais une performance future.

Le périmètre fonctionnel est organisé en trois modules complémentaires.

- **Module A - Explorateur ETF** : consultation du catalogue des ETF, recherche et comparaison graphique de deux ETF.
- **Module B - Simulateur DCA** : simulation d'une stratégie d'investissement mensuel sur un ETF, avec capital initial, versement mensuel, dates de début/fin et prise en compte du TER.
- **Module C - Régression linéaire** : analyse statistique d'un ETF sur une fenêtre de 3, 5, 10 ou 15 ans avec tendance, R², p-value, résidus et projection illustrative.



Figure 1 - Page d'accueil de l'application

II. NOTIONS FINANCIÈRES

A. ETF ET INVESTISSEMENT PASSIF

Un ETF, ou Exchange Traded Fund, est un fonds coté en bourse qui réplique la performance d'un indice. Acheter un ETF revient à s'exposer à un panier de titres plutôt qu'à une seule action. Par exemple, un ETF MSCI World donne accès à un grand nombre d'entreprises internationales, tandis qu'un ETF S&P 500 suit les grandes capitalisations américaines.

L'investissement passif repose sur l'idée qu'il est difficile de battre durablement le marché. Au lieu de chercher les meilleures actions ou le meilleur moment pour entrer, l'investisseur accepte la performance moyenne d'un indice, avec des frais généralement plus faibles que ceux de la gestion active.

Dans notre application, les ETF sont stockés dans la table etf avec leurs principales caractéristiques : ticker, nom, indice répliqué, gestionnaire, TER et éligibilité PEA. Ces informations sont affichées dans l'explorateur ETF et utilisées par les modules de simulation.

B. DCA : Dollar Cost Averaging

Le DCA, ou investissement programmé, consiste à investir une somme fixe à intervalles réguliers. Dans notre projet, l'intervalle retenu est mensuel. L'objectif est de lisser le prix d'achat moyen : quand le prix de l'ETF baisse, le versement mensuel permet d'acheter davantage de parts ; quand le prix augmente, il permet d'en acheter moins.

Cette stratégie est particulièrement adaptée à une logique de long terme, car elle limite la dépendance au point d'entrée initial. Elle ne supprime pas le risque de marché, mais elle évite de concentrer tout le capital sur une seule date.

C. TER

Le TER, ou Total Expense Ratio, représente les frais annuels d'un fonds. Même lorsqu'il paraît faible, son effet se cumule dans le temps. Un écart de quelques dixièmes de pourcentage peut devenir significatif sur une période longue, car les frais diminuent la valeur capitalisée du portefeuille.

Le simulateur applique le TER de manière mensuelle en divisant le taux annuel par 12. Cette modélisation reste une approximation pédagogique, mais elle permet de visualiser l'écart entre une valeur de portefeuille avec frais et une valeur théorique sans frais.

D. Backtesting

Le backtesting consiste à tester une stratégie sur des données historiques. Dans notre cas, l'application rejoue mois par mois une stratégie DCA sur les prix historiques d'un ETF. Le résultat indique ce qui se serait passé dans le passé pour des paramètres donnés.

Le backtesting a une limite importante : il décrit le passé, mais ne prédit pas le futur. Les crises, les changements de taux, l'inflation, la composition des indices ou le comportement des investisseurs peuvent modifier les performances futures.

E. Régression linéaire appliquée à un ETF

La régression linéaire permet d'estimer une tendance moyenne entre une variable explicative et une variable expliquée. Dans le module C, la variable explicative est le temps, représenté par un index de jours de cotation, et la variable expliquée est le prix ajusté de clôture de l'ETF.

Ce modèle doit être interprété avec prudence : les prix financiers ne suivent pas parfaitement une droite, les résidus ne sont pas toujours aléatoires et les périodes de crise peuvent fortement influencer les résultats.

III. ARCHITECTURE ET CHOIX TECHNIQUE

1. Méthode de travail

Le travail a d'abord commencé par une phase commune de mise en place du projet. Nous avons travaillé tous les trois sur la création de la base technique : initialisation du dépôt Git, organisation des dossiers, création des fichiers de configuration comme le `.env` et le `.gitignore`, mise en place de la base de données, ainsi que rédaction du `README` pour documenter l'installation et le lancement du projet.

Une fois cette base commune établie, chaque membre du groupe a travaillé sur un module précis du projet. Cette répartition nous a permis de paralléliser le développement des différentes fonctionnalités, tout en gardant une organisation claire. Les modules ont ensuite été intégrés progressivement afin d'obtenir une version finale cohérente et fonctionnelle de l'application.

Pour limiter les conflits et conserver une version stable du projet, nous avons utilisé des branches Git dédiées, soit par personne, soit par module. Les développements étaient réalisés sur ces branches, puis intégrés progressivement à la branche principale après vérification. Cette méthode nous a permis de travailler en parallèle tout en gardant une base commune cohérente et fonctionnelle.

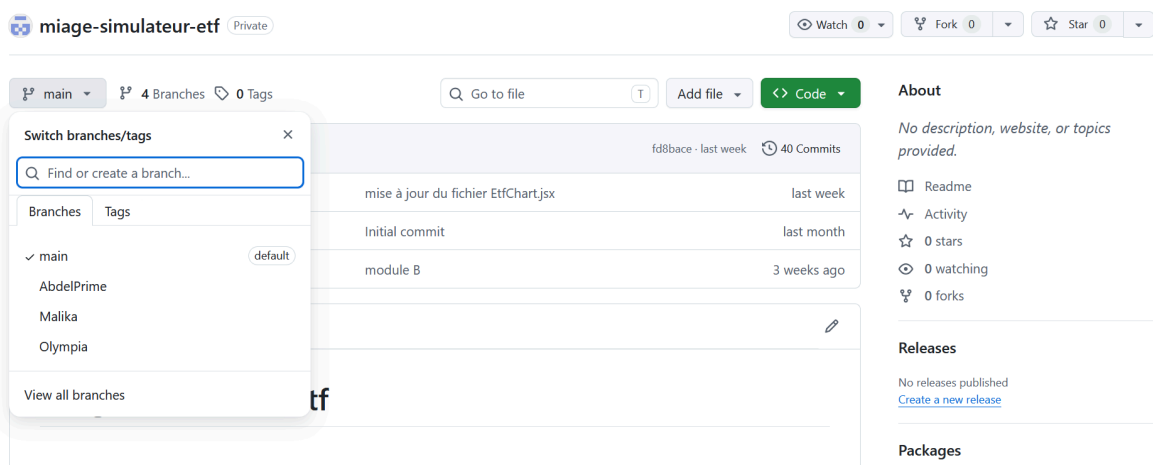


Figure 2 - Organisation Git du groupe

2. Architecture générale

L'application repose sur une architecture web en trois couches : **frontend**, **backend** et **base de données**. Cette organisation permet de séparer l'affichage, les traitements et le stockage des données, ce qui rend le projet plus clair et plus facile à maintenir.

Le **frontend**, développé avec **React**, correspond à l'interface utilisateur. Il permet de consulter les ETF, saisir les paramètres des simulations et afficher les résultats. React a été choisi car il facilite la création d'interfaces dynamiques et organisées en composants.

Le **backend**, développé avec **FastAPI**, fait le lien entre l'interface et la base de données. Il expose les routes REST, reçoit les requêtes du frontend, exécute les calculs financiers et renvoie les résultats au format JSON. FastAPI est adapté car il est léger, rapide et utilise Python, un langage pratique pour le traitement de données et les calculs statistiques.

La **base de données PostgreSQL** stocke les ETF, les cours historiques, les simulations et les résultats de régression. Ce choix est pertinent car le projet repose sur des données structurées et reliées entre elles.

La communication entre les différentes couches se fait de manière structurée. Le frontend envoie des requêtes HTTP au backend, qui traite la demande puis renvoie une réponse au format JSON. De son côté, le backend interagit avec PostgreSQL à l'aide de requêtes SQL pour lire ou enregistrer les données. Cette séparation permet d'éviter que le frontend accède directement à la base de données, ce qui améliore la sécurité et centralise les règles de gestion dans le backend.

Cette architecture est donc adaptée au projet, car elle permet de répartir clairement les rôles, de faciliter le travail en équipe et de faire évoluer l'application plus facilement.

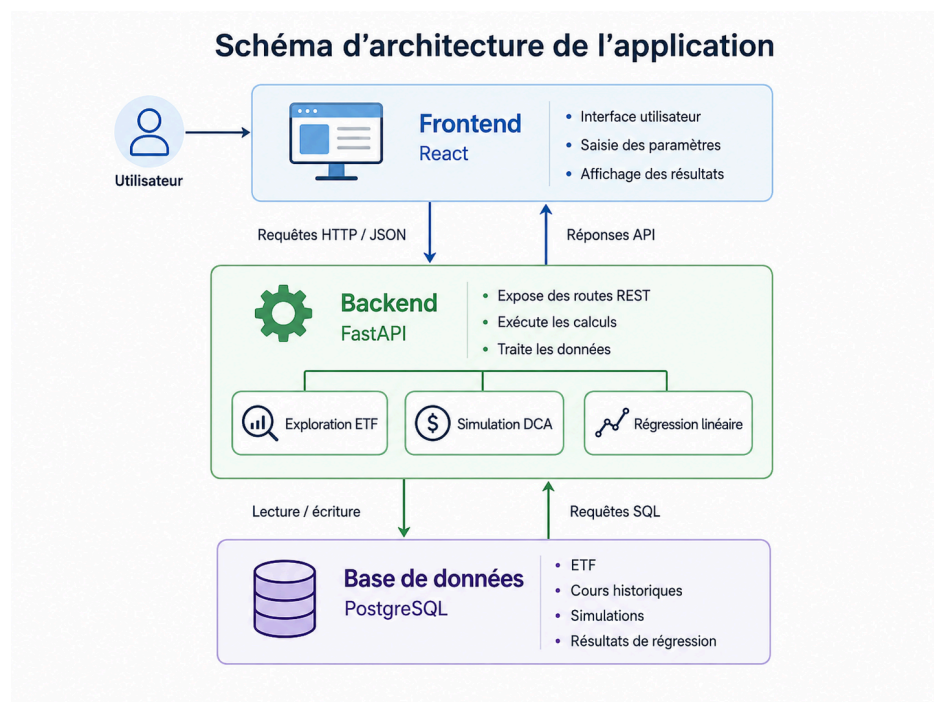


Figure 3 - Architecture générale du simulateur

3. Modèle de données

Le modèle de données est centré sur la table etf. Chaque cours historique est rattaché à un ETF, chaque simulation est rattachée à un ETF et produit plusieurs lignes dans résultat simulation. Les résultats de régression sont également liés à un ETF.

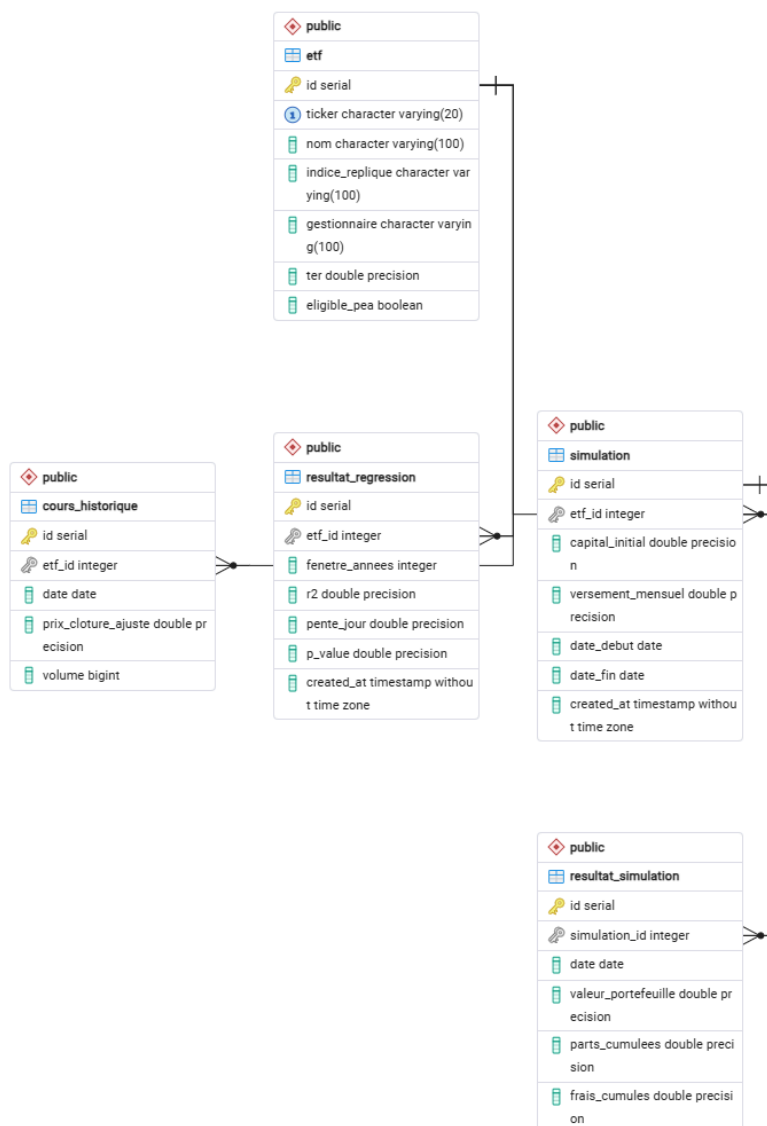


Figure 4 - Modèle de données du projet

IV. PIPELINE DE DONNÉE

1. Sources utilisées

Les cours historiques sont récupérés à l'aide de la bibliothèque yfinance. Le script backend/scripts/fetch_etf.py télécharge les données à partir du 1er janvier 2015 et les insère dans PostgreSQL. Les ETF utilisés dans le script d'import sont les suivants :

Ticker	Nom	Indice répliqué	Gestionnaire	TER	PEA
CW8.PA	Amundi MSCI World	MSCI World	Amundi	0,38 %	Oui
500.PA	Amundi S&P 500	S&P 500	Amundi	0,15 %	Oui
ESE.PA	iShares MSCI Europe	MSCI Europe	iShares	0,12 %	Non
OBLI.PA	Lyxor Obligations État Euro	MTS Italy	Lyxor	0,17 %	Non

2. Processus ETL

Le processus ETL peut être résumé en trois étapes : extraction, transformation et chargement. L'extraction consiste à télécharger les données depuis Yahoo Finance via yfinance. La transformation consiste à convertir les dates, le prix de clôture ajusté et le volume dans un format compatible avec PostgreSQL. Le chargement insère d'abord les ETF, puis les lignes de cours dans la table cours_historique.

- Connexion à PostgreSQL avec DATABASE_URL.
- Insertion de chaque ETF dans la table etf avec ON CONFLICT pour éviter les doublons.
- Téléchargement des prix historiques avec yf.download(ticker, start='2015-01-01', auto_adjust=True).
- Préparation d'une liste de records contenant etf_id, date, prix de clôture ajusté et volume.
- Insertion en lot avec executemany afin d'améliorer les performances.
- Commit après chaque ETF pour sécuriser l'import progressif.

3. Qualité et cohérence des données

Plusieurs choix améliorent la cohérence des données. Le modèle relationnel évite la duplication des informations ETF : les cours et les résultats ne stockent que l'identifiant de l'ETF. Les requêtes de simulation filtrent les prix nuls et sélectionnent le premier prix disponible de chaque mois avec DISTINCT ON (DATE_TRUNC('month', date)).

Pour le DCA, l'utilisation d'un point mensuel est cohérente avec un versement mensuel. Pour la régression, les données quotidiennes sont conservées afin d'avoir davantage de points et une estimation statistique plus fine.

4. Fréquence de mise à jour

Dans l'état actuel du projet, l'import est déclenché par un script manuel. Une amélioration consisterait à automatiser l'exécution de `fetch_etf.py`, par exemple une fois par jour ou une fois par semaine, afin de maintenir les cours historiques à jour.

Une fréquence quotidienne serait pertinente pour les cours de marché, mais une mise à jour hebdomadaire peut suffire dans le cadre pédagogique du projet. L'essentiel est d'avoir une date de dernière mise à jour visible et de prévenir l'utilisateur si les données sont anciennes.

V. SIMULATEUR DCA

1. Objectif du module

Le simulateur DCA permet à l'utilisateur de choisir un ETF, un capital initial, un versement mensuel, une période de début et de fin, ainsi qu'un TER. L'objectif est d'obtenir une simulation complète de la valeur du portefeuille mois par mois.

L'interface React envoie les paramètres au backend via la route POST `/simulation/`. Le backend vérifie que l'ETF existe, valide les dates, récupère le TER et exécute le calcul. Les résultats sont ensuite sauvegardés en base et renvoyés au frontend.

The image shows a web application interface for a DCA (Dollar Cost Averaging) simulator. The interface is titled "Portefeuille Passif" and has a navigation bar with links to "Accueil", "ETF", "Simulateur" (which is highlighted), "Régression", and "M2 MIAGE". The main heading is "Configurez votre stratégie". Below this, there is a text prompt: "Sélectionnez un ETF, définissez votre capital de départ et votre versement mensuel, puis lancez la simulation pour voir comment votre portefeuille aurait évolué." The form contains several input fields: "ETF" (a dropdown menu showing "CW8.PA — Amundi MSCI World"), "Capital de départ (€)" (a text input with "1000"), "Versement mensuel (€)" (a text input with "200"), "Début" and "Fin" (two date pickers showing "01 / 01 / 2015" and "31 / 12 / 2024" respectively), and "TER (frais annuels, ex: 0.0038 = 0.38%)" (a text input with "0.0038"). At the bottom of the form is a blue button labeled "Lancer la simulation".

Figure 6 - Formulaire du simulateur DCA

2. Algorithme de calcul

Le service `dca_service.py` récupère le premier cours disponible de chaque mois sur la période sélectionnée. Ensuite, il parcourt les mois dans l'ordre chronologique et applique les opérations de la stratégie DCA.

- 1. Initialiser le nombre de parts cumulées à zéro.
- 2. Pour le premier mois, investir le capital initial plus le versement mensuel.
- 3. Pour les mois suivants, investir uniquement le versement mensuel.
- 4. Calculer les nouvelles parts achetées : montant investi / prix du mois.
- 5. Ajouter ces parts au total cumulé.
- 6. Calculer la valeur brute du portefeuille : parts cumulées x prix.
- 7. Appliquer le TER mensuel : TER annuel / 12.
- 8. Calculer la valeur après frais et stocker les métriques mensuelles.
- 9. Mettre à jour une comparaison avec un Livret A à 3 % annuel, modélisé mensuellement.

Métriques renvoyées par l'API :

- `capital_total_verse`,
- `valeur_finale`,
- `valeur_finale_sans_frais`,
- `gain_net`,
- `impact_frais_total`,
- `CAGR`, `nombre_mois`,
- `valeur_livret_a`

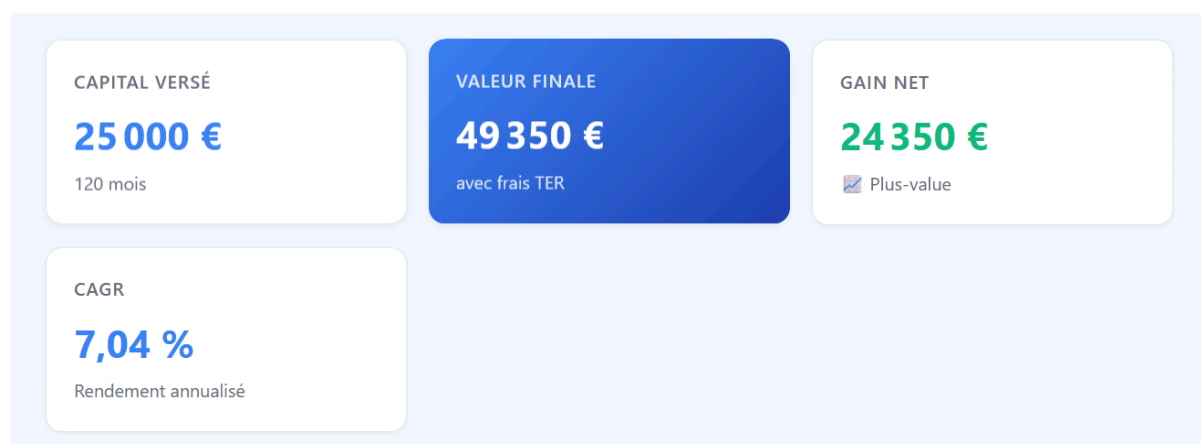


Figure 7 - Résultats DCA pour CW8.PA

3. Sauvegarde des résultats

Après le calcul, une ligne est créée dans la table `simulation` avec les paramètres de départ. Les résultats mensuels sont insérés dans `resultat_simulation`. Cette séparation est utile car une simulation possède plusieurs points temporels : une ligne pour les paramètres et plusieurs lignes pour l'évolution du portefeuille.

La route `GET /simulation/{simulation_id}` permet ensuite de récupérer une simulation persistée et ses résultats, ce qui facilite la consultation ou la réutilisation des simulations.

4. Cas de test

Pour répondre à l'attendu du projet, le simulateur doit être testé sur au moins deux ETF. Les ETF choisis sont les suivants :

Cas	ETF	Paramètres de test	Résultats à reporter
Test 1	CW8.PA - Amundi MSCI World	Capital initial 1 000 €, versement 200 €/mois, 01/01/2015 -> 31/12/2024, TER 0,38 %	Valeur finale, gain net, CAGR, impact des frais, écart avec Livret A
Test 2	500.PA - Amundi S&P 500	Capital initial 1 000 €, versement 200 €/mois, 01/01/2015 -> 31/12/2024, TER 0,15 %	Valeur finale, gain net, CAGR, impact des frais, écart avec Livret A

- Test 1 :



Figure 7 - Résultats DCA pour CW8.PA



- Test 2



Figure 8 - Résultats DCA pour 500.PA



Figure 9 - Évolution mensuelle du portefeuille pour 500.PA

5. Analyse de l'impact des frais

Le simulateur affiche une comparaison entre la valeur finale avec frais et la valeur finale théorique sans frais. Cette différence correspond à l'impact cumulé du TER. Le résultat met en évidence un effet souvent sous-estimé : même avec un TER faible, les frais diminuent progressivement le nombre de parts et donc la valeur finale du portefeuille.

Le module compare aussi la simulation avec un Livret A modélisé à 3 % par an. Cette comparaison permet de distinguer un placement risqué, exposé au marché, d'un placement réglementé et moins volatil. Elle doit rester pédagogique : le niveau du taux du Livret A varie dans le temps et la simulation utilise ici une hypothèse fixe.

- Test 1

Impact des frais TER

Scénario	Valeur finale	Différence
Sans frais (TER = 0%)	50 502 € —	
Avec frais réels (TER ETF)	49 350 €	-1 153 €
Livret A (3% / an, sans risque)	29 298 €	+20 052 €

💡 Sur le long terme, même un TER faible représente une perte importante. Les ETF passifs (TER ~0.20%) sont beaucoup plus rentables que les fonds actifs (TER ~1.5%).

Figure 10 - Impact des frais TER CW8.PA

- Test 2

Impact des frais TER

Scénario	Valeur finale	Différence
Sans frais (TER = 0%)	60 241 € —	
Avec frais réels (TER ETF)	59 674 €	-567 €
Livret A (3% / an, sans risque)	29 298 €	+30 376 €

💡 Sur le long terme, même un TER faible représente une perte importante. Les ETF passifs (TER ~0.20%) sont beaucoup plus rentables que les fonds actifs (TER ~1.5%).

Figure 11 - Impact des frais TER 500.PA

VI. RÉGRESSION LINÉAIRE

1. Objectif du module

Le module de régression linéaire vise à analyser la tendance historique d'un ETF. Il ne cherche pas à prédire précisément le prix futur, mais à visualiser une tendance moyenne sur une fenêtre temporelle choisie par l'utilisateur.

L'utilisateur sélectionne un ETF et une fenêtre d'analyse de 3, 5, 10 ou 15 ans. Le frontend appelle la route POST /regression/ puis affiche les métriques, la droite de régression, l'intervalle de confiance et les résidus.

Figure 11 - Formulaire du module de régression

2. Méthodologie statistique

Le backend construit une variable X correspondant au rang du jour de cotation dans la fenêtre d'analyse : 0, 1, 2, etc. La variable Y correspond au prix de clôture ajusté. La fonction `scipy.stats.linregress` calcule la pente, l'intercept, le coefficient de corrélation, la p-value et l'erreur standard.

Équation estimée :

$$Y = \text{beta_0} + \text{beta_1} X + \text{epsilon}$$

- Y : prix ajusté de l'ETF
- X : index temporel
- beta_1 : pente moyenne par jour de cotation

À partir de ces valeurs, le module calcule R2, les prix prédits, les résidus et une pente annualisée. Une projection illustrative sur 252 jours de cotation est également produite à partir de la droite. Le frontend la présente comme une extension théorique, et non comme une prévision fiable.

3. Indicateurs interprétés

Indicateur	Définition	Interprétation
R2	Part de la variation du prix expliquée par la tendance linéaire	Plus R2 est élevé, plus la droite suit la tendance historique ; cela ne garantit pas une bonne prédiction future.
Pente journalière	Variation moyenne du prix par jour de cotation	Une pente positive indique une tendance haussière moyenne sur la période.
Pente annualisée	Approximation de la tendance sur une année	Permet une lecture plus intuitive que la pente journalière.

P-value	Test de significativité de la pente	Si elle est inférieure à 5 %, la pente est considérée comme statistiquement significative.
Résidus	Écart entre prix réel et prix prédit	Permet de voir les crises, bulles, corrections ou phases où la droite explique mal les prix.
IC 95 %	Intervalle de confiance autour de la tendance	Plus l'intervalle est large, plus l'incertitude de l'estimation est élevée.

4. Résultats à présenter sur deux ETF

- Test 1 : [CW8.PA](#) sur 10 ans

Synthèse de la régression

ETF : **CW8.PA** · Fenêtre : **10 ans**



Figure 12- Régression linéaire sur CW8.PA

Analyse des résidus

Les résidus correspondent à la différence entre le cours réel et le cours prédit.



Figure 13 - Résidus de la régression pour [CW8.PA](#)

- Test 1 : 500.[PA](#) sur 10 ans



Figure 14- Régression linéaire sur [CW8.PA](#)



Figure 15 - Résidus de la régression pour [CW8.PA](#)

5. Interprétation critique

Même si le modèle de régression linéaire peut afficher un R^2 élevé, par exemple autour de 0,91, cela ne signifie pas automatiquement que le modèle est capable de prédire correctement les cours futurs. Dans le cas d'un ETF dont le prix augmente globalement sur une longue période, un R^2 élevé est en réalité assez attendu. En effet, la régression capte surtout la tendance générale de la série temporelle : si les prix montent progressivement dans le temps, une droite peut expliquer une grande partie de cette évolution passée.

Cependant, cela ne veut pas dire que le modèle est réellement prédictif. Le marché financier ne suit pas une progression parfaitement linéaire : il est soumis à des variations, des crises, des corrections, des annonces économiques ou encore des changements de contexte. La régression donne donc une lecture simplifiée de la tendance passée, mais elle ne permet pas d'anticiper avec certitude les cours futurs.

L'analyse des résidus est donc essentielle. Les résidus correspondent aux écarts entre les valeurs réellement observées et les valeurs estimées par le modèle. Lorsqu'ils sont importants, ils montrent que certains mouvements du marché ne sont pas expliqués par la tendance linéaire. Cela rappelle que les prix financiers intègrent de nombreuses informations imprévisibles et que les marchés peuvent réagir rapidement à de nouveaux événements.

Ainsi, un R^2 élevé ne doit pas être interprété comme une preuve que "le modèle est bon" au sens prédictif. Il indique surtout que le modèle décrit correctement une tendance historique globale. Les résidus, eux, montrent les limites du modèle et mettent en évidence l'incertitude propre aux marchés financiers. Cette observation rejoint l'idée d'efficacité des marchés : si les informations disponibles sont déjà intégrées dans les prix, il devient difficile de prévoir systématiquement les variations futures à partir des seules données passées.

6. Limites du modèle de régression

- La variable temps est la seule variable explicative ; le modèle ignore les taux, l'inflation, les résultats des entreprises ou la composition des indices.
- Les prix financiers sont volatils et ne suivent pas parfaitement une droite.
- Les résidus peuvent être autocorrélés, ce qui limite l'interprétation classique de la régression.
- La projection à 12 mois est une extrapolation linéaire et doit rester illustrative.
- Le choix de la fenêtre temporelle influence fortement les résultats. Une fenêtre commençant après une crise peut donner une tendance différente d'une fenêtre incluant la crise.

VII. CONCLUSION

Ce projet nous a permis de relier des notions financières, statistiques et informatiques dans une application complète. Le frontend rend l'outil accessible à l'utilisateur, le backend centralise les calculs et la base PostgreSQL organise les données nécessaires aux simulations et aux analyses.

Le simulateur DCA permet d'illustrer concrètement l'effet des versements réguliers, du temps et des frais sur un portefeuille passif. Le module de régression linéaire complète cette approche en apportant une lecture statistique des tendances historiques. Il permet d'observer une évolution passée, tout en rappelant que les résultats obtenus ne peuvent pas être interprétés comme une prédiction certaine de la performance future.

Malgré son intérêt pédagogique, le projet présente plusieurs limites. La base de données doit être alimentée et maintenue à jour pour que les résultats restent pertinents. Le TER est appliqué de manière simplifiée sous forme mensuelle, alors que la réalité comptable d'un ETF peut être plus complexe. De plus, la comparaison avec le Livret A repose sur un taux fixe, alors que ce taux varie dans le temps. Les résultats dépendent également fortement de la période choisie et ne constituent donc pas une recommandation financière.

Certaines limites techniques ont aussi été identifiées. Le projet gagnerait à intégrer un fichier `requirements.txt` ou `pyproject.toml` afin de fiabiliser l'installation du backend et d'éviter des erreurs liées aux dépendances, comme l'absence de `uvicorn`. De plus, une incohérence de casse dans l'import du fichier `regressionAPI.js` peut fonctionner sous Windows, mais poser problème sur un environnement Linux sensible à la casse.

Avec davantage de temps, plusieurs améliorations pourraient être apportées. Il serait pertinent d'automatiser la mise à jour des cours avec une tâche planifiée, d'ajouter des tests unitaires sur l'algorithme DCA et la régression, ou encore de créer une page d'administration indiquant la dernière date de mise à jour des données. L'application pourrait aussi permettre de comparer plusieurs stratégies d'investissement, comme le DCA, l'investissement initial unique ou les versements variables. Enfin, l'export des résultats en CSV ou en PDF faciliterait l'analyse et la réutilisation des simulations.

Sur le plan financier, ce projet met en évidence l'intérêt de l'investissement passif comme approche de long terme, grâce à la diversification, aux frais réduits et à la simplicité de mise en place. Il montre aussi que la performance ne peut pas être séparée du risque ni de l'horizon de placement. Une stratégie DCA peut être rationnelle, mais elle ne protège pas totalement contre les baisses de marché.

Enfin, ce projet rappelle que les outils statistiques sont utiles pour comprendre les tendances, mais qu'ils doivent être interprétés avec prudence. Une courbe ou une droite de tendance peut donner une impression de certitude, alors que les marchés financiers restent par nature incertains.

Annexe

Lien GitHub : <https://github.com/abouzi1/miage-simulateur-etf>

Lien de l'application : <https://miage-simulateur-etf.vercel.app/>

Documentation API image

Simulateur de Portefeuille Passif 1.0.0 OAS 3.1

/openapi.json

API du projet DATA M2 MIAGE - Modules B et C

Simulation DCA ^

POST /simulation/ Lancer une simulation DCA v

GET /simulation/{simulation_id} Récupérer une simulation existante v

Regression ^

POST /regression/ Lancer Regression v

GET /regression/{ticker} Get Regression v

explorateur ETF ^

GET /etfs/ Get Etfs v

GET /etfs/{etf_id}/historique Get Etf Historique v

Health ^

GET / Root v

GET /health Health 📄 v